ABSTRACT

BookNest is a book tracking application built with Flutter and Dart for Android and the web (Chrome). It is inspired by Goodreads but uses a calmer, editorial design with a deep forest-green, ivory and bronze colour palette, and supports both light and dark themes.

The app lets a reader search for books by title, author or keyword through the public Open Library API, view book covers and details, and build a personal library. Each book can be marked as *Want to Read*, *Currently Reading* or *Read*, marked as a favorite, rated from one to five stars, given a personal review, and tracked with a reading-progress percentage. A yearly reading goal shows how many books have been completed.

The application is organised into four screens reachable from a bottom navigation bar: **Home** (goal progress, continue reading, suggestions, recently added and a sample quotation card), **Discover** (debounced search and genre chips), **My Books** (status tabs, sorting, status changes and removal with confirmation) and **Profile** (name, statistics derived from the library, goal and theme preference).

All user data is stored locally using the *shared_preferences* package, so no account, server or API key is needed. State is managed with a single *ChangeNotifier* shared through an *InheritedNotifier*, which keeps every screen consistent without extra state-management packages. The app works without an internet connection by falling back to built-in sample books, handles missing covers and corrupt saved data gracefully, and prevents duplicate library entries. The project passes *flutter analyze*, an automated test suite (library logic, persistence, corrupt data, navigation and layout at narrow and wide widths) and a successful web build.

Technologies used

- Flutter (Material 3) and Dart
- Open Library Search and Works APIs (book data, descriptions and covers)
- http package for network requests
- shared_preferences package for local persistence

FILE STRUCTURE

```
booknest
|-- .dart_tool
|-- .idea
|-- android
|-- build
|-- ios
|-- lib
|   |-- api.dart
|   |-- book.dart
|   |-- details.dart
|   |-- discover.dart
|   |-- home.dart
|   |-- library.dart
|   |-- main.dart
|   |-- my_books.dart
|   |-- profile.dart
|   `-- widgets.dart
|-- linux
|-- macos
|-- pictures
|-- test
|   `-- widget_test.dart
|-- web
|-- windows
|-- .gitignore
|-- .metadata
|-- analysis_options.yaml
|-- booknest.iml
|-- pubspec.lock
|-- pubspec.yaml
`-- README.md
```

Purpose of the main source files

- **main.dart** – App entry point, theme mode and bottom navigation (tabs keep their state).
- **book.dart** – Book model, reading statuses, JSON conversion, sample books and quotes.
- **library.dart** – Library state: loading/saving, add, update, remove, counts and sorting.
- **api.dart** – Open Library search and description requests.
- **widgets.dart** – Design tokens, light/dark themes and reusable widgets.
- **home.dart, discover.dart, details.dart, my_books.dart, profile.dart** – The screens of the app.
- **widget_test.dart** – Unit and widget tests.

SOURCE CODE

main.dart

```
import 'package:flutter/material.dart';

import 'discover.dart';
import 'home.dart';
import 'library.dart';
import 'my_books.dart';
import 'profile.dart';
import 'widgets.dart';

void main() {
  runApp(const BookNestApp());
}

class BookNestApp extends StatefulWidget {
  const BookNestApp({super.key});

  @override
  State<BookNestApp> createState() => _BookNestAppState();
}

class _BookNestAppState extends State<BookNestApp> {
  final library = Library();

  @override
  void initState() {
    super.initState();
    library.load();
  }

  @override
  void dispose() {
    library.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return LibraryScope(
      library: library,
      // Rebuild the MaterialApp when the theme preference changes.
      child: ListenableBuilder(
        listenable: library,
        builder: (context, _) => MaterialApp(
          title: 'BookNest',
          debugShowCheckedModeBanner: false,
          theme: buildTheme(Brightness.light),
          darkTheme: buildTheme(Brightness.dark),
          themeMode: library.themeMode,
          home: const AppShell(),
        ),
      ),
    );
  }
}

class AppShell extends StatefulWidget {
  const AppShell({super.key});

  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  int _tab = 0;
  bool _warned = false;

  void _goTo(int index) => setState(() => _tab = index);

  @override
  Widget build(BuildContext context) {
    final lib = context.lib;
    if (lib.saveFailed && !_warned) {
      _warned = true;
```

```
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (mounted) {
          toast(
            context,
            'Could not save data on this device. Changes will last for this session only.',
          );
        }
      });
    }
    if (!lib.loaded) {
      return const Scaffold(body: Center(child: CircularProgressIndicator()));
    }
    return Scaffold(
      // IndexedStack keeps each tab's scroll position and search results.
      body: SafeArea(
        bottom: false,
        child: IndexedStack(
          index: _tab,
          children: [
            HomeScreen(onGoTo: _goTo),
            const DiscoverScreen(),
            MyBooksScreen(onGoTo: _goTo),
            const ProfileScreen(),
          ],
        ),
      ),
      bottomNavigationBar: DecoratedBox(
        decoration: BoxDecoration(
          border: Border(
            top: BorderSide(
              color: Theme.of(context).colorScheme.outlineVariant,
            ),
          ),
        ),
        child: NavigationBar(
          selectedIndex: _tab,
          onDestinationSelected: _goTo,
          destinations: const [
            NavigationDestination(
              icon: Icon(Icons.home_outlined),
              selectedIcon: Icon(Icons.home),
              label: 'Home',
            ),
            NavigationDestination(
              icon: Icon(Icons.explore_outlined),
              selectedIcon: Icon(Icons.explore),
              label: 'Discover',
            ),
            NavigationDestination(
              icon: Icon(Icons.collections_bookmark_outlined),
              selectedIcon: Icon(Icons.collections_bookmark),
              label: 'My Books',
            ),
            NavigationDestination(
              icon: Icon(Icons.person_outline),
              selectedIcon: Icon(Icons.person),
              label: 'Profile',
            ),
          ],
        ),
      ),
    );
  }
}
```

book.dart

```
enum ReadingStatus {
  wantToRead('Want to Read'),
  reading('Currently Reading'),
  read('Read');

  const ReadingStatus(this.label);
  final String label;
}

/// A book. Search results only use the first few fields; the rest hold the
/// user's personal data once the book is saved to the library.
class Book {
  const Book({
    required this.id,
    required this.title,
    required this.author,
    this.coverUrl,
    this.year,
    this.publisher,
    this.description,
    this.genres = const [],
    this.status = ReadingStatus.wantToRead,
    this.favorite = false,
    this.rating = 0,
    this.review = '',
    this.progress = 0,
    this.addedAt,
    this.isSample = false,
  });

  final String id;
  final String title;
  final String author;
  final String? coverUrl;
  final int? year;
  final String? publisher;
  final String? description;
  final List<String> genres;

  // Personal data
  final ReadingStatus status;
  final bool favorite;
  final int rating; // 0 = not rated
  final String review;
  final int progress; // 0-100
  final DateTime? addedAt;
  final bool isSample;

  Book copyWith({
    String? description,
    ReadingStatus? status,
    bool? favorite,
    int? rating,
    String? review,
    int? progress,
    DateTime? addedAt,
    bool? isSample,
  }) => Book(
    id: id,
    title: title,
    author: author,
    coverUrl: coverUrl,
    year: year,
    publisher: publisher,
    description: description ?? this.description,
    genres: genres,
    status: status ?? this.status,
    favorite: favorite ?? this.favorite,
    rating: rating ?? this.rating,
    review: review ?? this.review,
    progress: progress ?? this.progress,
    addedAt: addedAt ?? this.addedAt,
    isSample: isSample ?? this.isSample,
  );

  /// Used to spot the same book even if it has a different id.
  String get matchKey => '${_norm(title)}|${_norm(author)}';
```

```
  static String _norm(String s) =>
      s.toLowerCase().replaceAll(RegExp(r'[^a-z0-9]'), '');
  Map<String, dynamic> toJson() => {
    'id': id,
    'title': title,
    'author': author,
    'coverUrl': coverUrl,
    'year': year,
    'publisher': publisher,
    'description': description,
    'status': status.name,
    'favorite': favorite,
    'rating': rating,
    'review': review,
    'progress': progress,
    'addedAt': addedAt?.toIso8601String(),
    'isSample': isSample,
  };

  /// Returns null if the stored data is unusable.
  static Book? fromJson(Object? raw) {
    if (raw is! Map) return null;
    final id = raw['id'], title = raw['title'];
    if (id is! String || title is! String) return null;
    return Book(
      id: id,
      title: title,
      author: raw['author'] is String ? raw['author'] : 'Unknown author',
      coverUrl: raw['coverUrl'] is String ? raw['coverUrl'] : null,
      year: raw['year'] is int ? raw['year'] : null,
      publisher: raw['publisher'] is String ? raw['publisher'] : null,
      description: raw['description'] is String ? raw['description'] : null,
      status: ReadingStatus.values.firstWhere(
        (s) => s.name == raw['status'],
        orElse: () => ReadingStatus.wantToRead,
      ),
      favorite: raw['favorite'] == true,
      rating: raw['rating'] is int ? (raw['rating'] as int).clamp(0, 5) : 0,
      review: raw['review'] is String ? raw['review'] : '',
      progress: raw['progress'] is int
          ? (raw['progress'] as int).clamp(0, 100)
          : 0,
      addedAt: DateTime.tryParse('${raw['addedAt']}'),
      isSample: raw['isSample'] == true,
    );
  }
}

String _isbn(String isbn) =>
    'https://covers.openlibrary.org/b/isbn/$isbn-M.jpg?default=false';

/// Built-in sample books: used offline and for "Recommended" suggestions.
final List<Book> sampleBooks = [
  Book(
    id: 'sample:hobbit',
    title: 'The Hobbit',
    author: 'J.R.R. Tolkien',
    year: 1937,
    coverUrl: _isbn('9780547928227'),
    genres: const ['fantasy', 'fiction'],
  ),
  Book(
    id: 'sample:dune',
    title: 'Dune',
    author: 'Frank Herbert',
    year: 1965,
    coverUrl: _isbn('9780441172719'),
    genres: const ['science fiction', 'fiction'],
  ),
  Book(
    id: 'sample:mockingbird',
    title: 'To Kill a Mockingbird',
    author: 'Harper Lee',
    year: 1960,
    coverUrl: _isbn('9780061120084'),
    genres: const ['fiction'],
  ),
  Book(
    id: 'sample:pride',
```

```
    title: 'Pride and Prejudice',
    author: 'Jane Austen',
    year: 1813,
    coverUrl: _isbn('9780141439518'),
    genres: const ['romance', 'fiction'],
  ),
  Book(
    id: 'sample:orient',
    title: 'Murder on the Orient Express',
    author: 'Agatha Christie',
    year: 1934,
    coverUrl: _isbn('9780062693662'),
    genres: const ['mystery', 'fiction'],
  ),
  Book(
    id: 'sample:1984',
    title: '1984',
    author: 'George Orwell',
    year: 1949,
    coverUrl: _isbn('9780451524935'),
    genres: const ['science fiction', 'fiction'],
  ),
  Book(
    id: 'sample:hailmary',
    title: 'Project Hail Mary',
    author: 'Andy Weir',
    year: 2021,
    coverUrl: _isbn('9780593135204'),
    genres: const ['science fiction'],
  ),
  Book(
    id: 'sample:gatsby',
    title: 'The Great Gatsby',
    author: 'F. Scott Fitzgerald',
    year: 1925,
    coverUrl: _isbn('9780743273565'),
    genres: const ['fiction', 'romance'],
  ),
  Book(
    id: 'sample:name',
    title: 'The Name of the Wind',
    author: 'Patrick Rothfuss',
    year: 2007,
    coverUrl: _isbn('9780756404741'),
    genres: const ['fantasy'],
  ),
  Book(
    id: 'sample:girl',
    title: 'The Girl with the Dragon Tattoo',
    author: 'Stieg Larsson',
    year: 2005,
    coverUrl: _isbn('9780307454546'),
    genres: const ['mystery'],
  ),
];

const sampleQuotes = [
  ('A reader lives a thousand lives before he dies.', 'George R.R. Martin'),
  ('So many books, so little time.', 'Frank Zappa'),
  ('Books are a uniquely portable magic.', 'Stephen King'),
  ('There is no friend as loyal as a book.', 'Ernest Hemingway'),
];
```

library.dart

```
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'book.dart';

enum SortOption { recent, title, rating }

/// The app's single source of truth: library + preferences, saved locally.
class Library extends ChangeNotifier {
  List<Book> books = [];
  String name = 'Reader';
  int goal = 12;
  ThemeMode themeMode = ThemeMode.system;
  bool saveFailed = false;
  bool loaded = false;

  SharedPreferences? _prefs;

  Future<void> load() async {
    try {
      final prefs = _prefs = await SharedPreferences.getInstance();
      name = prefs.getString('name') ?? name;
      goal = prefs.getInt('goal') ?? goal;
      themeMode = ThemeMode.values.firstWhere(
        (m) => m.name == prefs.getString('theme'),
        orElse: () => ThemeMode.system,
      );
      final raw = prefs.getString('books');
      if (raw != null) {
        try {
          for (final item in jsonDecode(raw) as List) {
            final b = Book.fromJson(item);
            if (b != null && !books.any((e) => e.id == b.id)) books.add(b);
          }
        } catch (_) {
          books = []; // corrupt data: start fresh
        }
      }
      // Older versions saved hand-written text as the description of sample
      // books. Descriptions should only come from Open Library, so drop it.
      books = [
        for (final b in books)
          if (b.id.startsWith('sample:') && b.description != null)
            Book.fromJson({...b.toJson(), 'description': null})!
          else
            b,
      ];
      if (!(prefs.getBool('seeded') ?? false)) {
        // First launch only: a few clearly-labelled sample books.
        final now = DateTime.now();
        books = [
          sampleBooks[0].copyWith(
            status: ReadingStatus.reading,
            progress: 40,
            addedAt: now,
            isSample: true,
          ),
          sampleBooks[1].copyWith(
            status: ReadingStatus.wantToRead,
            addedAt: now.subtract(const Duration(minutes: 1)),
            isSample: true,
          ),
          sampleBooks[2].copyWith(
            status: ReadingStatus.read,
            rating: 5,
            progress: 100,
            addedAt: now.subtract(const Duration(minutes: 2)),
            isSample: true,
          ),
        ];
        await prefs.setBool('seeded', true);
        await _save();
      }
    } catch (_) {
      saveFailed = true; // storage unavailable: app still works this session
```

```
  }
  loaded = true;
  notifyListeners();
}

Future<void> _save() async {
  final prefs = _prefs;
  if (prefs == null) return;
  try {
    await prefs.setString(
      'books',
      jsonEncode(books.map((b) => b.toJson()).toList()),
    );
    await prefs.setString('name', name);
    await prefs.setInt('goal', goal);
    await prefs.setString('theme', themeMode.name);
  } catch (_) {
    saveFailed = true;
  }
}

void _changed() {
  notifyListeners();
  _save();
}

/// The saved copy of [b], matched by id or by title + author.
Book? find(Book b) {
  for (final s in books) {
    if (s.id == b.id || s.matchKey == b.matchKey) return s;
  }
  return null;
}

bool contains(Book b) => find(b) != null;

/// Adds the book, or updates the existing copy. Never creates duplicates.
/// Returns true if it was newly added.
bool save(Book b) {
  final existing = find(b);
  if (existing == null) {
    books = [b.copyWith(addedAt: DateTime.now(), isSample: false), ...books];
  } else {
    books = [
      for (final s in books)
        if (identical(s, existing))
          b.copyWith(addedAt: existing.addedAt, isSample: false)
        else
          s,
    ];
  }
  _changed();
  return existing == null;
}

void remove(Book b) {
  final existing = find(b);
  books = books.where((s) => !identical(s, existing)).toList();
  _changed();
}

void setStatus(Book b, ReadingStatus status) {
  final s = find(b);
  if (s == null) return;
  save(
    s.copyWith(
      status: status,
      progress: status == ReadingStatus.read
          ? 100
          : (status == ReadingStatus.wantToRead ? 0 : s.progress),
    ),
  );
}

void setName(String v) {
  name = v.trim().isEmpty ? 'Reader' : v.trim();
  _changed();
}
void setGoal(int v) {
```

```
    goal = v.clamp(1, 1000);
    _changed();
  }
  void setTheme(ThemeMode m) {
    themeMode = m;
    _changed();
  }

  // Everything below is calculated from the stored books.
  List<Book> withStatus(ReadingStatus s) =>
      books.where((b) => b.status == s).toList();
  int get readCount => withStatus(ReadingStatus.read).length;
  int get favoriteCount => books.where((b) => b.favorite).length;
  double get goalProgress => (readCount / goal).clamp(0.0, 1.0);

  List<Book> sorted(List<Book> list, SortOption sort) {
    final out = [...list];
    switch (sort) {
      case SortOption.recent:
        out.sort(
          (a, b) =>
              (b.addedAt ?? DateTime(0)).compareTo(a.addedAt ?? DateTime(0)),
        );
      case SortOption.title:
        out.sort(
          (a, b) => a.title.toLowerCase().compareTo(b.title.toLowerCase()),
        );
      case SortOption.rating:
        out.sort((a, b) => b.rating.compareTo(a.rating));
    }
    return out;
  }
}
```

api.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import 'book.dart';

const _timeout = Duration(seconds: 10);

/// Searches Open Library. Throws if the network or server fails.
Future<List<Book>> searchBooks(String query) async {
  final uri = Uri.https('openlibrary.org', '/search.json', {
    'q': query,
    'limit': '30',
    'fields': 'key,title,author_name,cover_i,first_publish_year,publisher',
  });
  final res = await http.get(uri).timeout(_timeout);
  if (res.statusCode != 200) {
    throw Exception('Server returned ${res.statusCode}');
  }
  final docs = (jsonDecode(res.body) as Map)['docs'];
  if (docs is! List) return [];
  final books = <Book>[];
  for (final d in docs) {
    if (d is! Map || d['key'] is! String || d['title'] is! String) continue;
    final authors = d['author_name'];
    final publishers = d['publisher'];
    books.add(
      Book(
        id: d['key'],
        title: d['title'],
        author: authors is List && authors.isNotEmpty
            ? '${authors.first}'
            : 'Unknown author',
        coverUrl: d['cover_i'] is int
            ? 'https://covers.openlibrary.org/b/id/${d['cover_i']}-M.jpg?default=false'
            : null,
        year: d['first_publish_year'] is int ? d['first_publish_year'] : null,
        publisher: publishers is List && publishers.isNotEmpty
            ? '${publishers.first}'
            : null,
      ),
    );
  }
  return books;
}

/// Fetches a work's description, or null if there isn't one.
Future<String?> fetchDescription(String workKey) async {
  if (!workKey.startsWith('/works/')) return null;
  final res = await http
      .get(Uri.https('openlibrary.org', '$workKey.json'))
      .timeout(_timeout);
  if (res.statusCode != 200) return null;
  final d = (jsonDecode(res.body) as Map)['description'];
  final text = d is String ? d : (d is Map ? d['value'] : null);
  if (text is! String || text.trim().isEmpty) return null;
  return text.split('----').first.trim();
}
```

home.dart

```
import 'package:flutter/material.dart';

import 'book.dart';
import 'details.dart';
import 'library.dart';
import 'widgets.dart';

const _weekdays = [
  'Monday',
  'Tuesday',
  'Wednesday',
  'Thursday',
  'Friday',
  'Saturday',
  'Sunday',
];
const _months = [
  'January',
  'February',
  'March',
  'April',
  'May',
  'June',
  'July',
  'August',
  'September',
  'October',
  'November',
  'December',
];

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key, required this.onGoTo});
  final ValueChanged<int> onGoTo;

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _quote = 0;

  String _greeting() {
    final h = DateTime.now().hour;
    return h < 12
        ? 'Good morning'
        : (h < 18 ? 'Good afternoon' : 'Good evening');
  }

  String _today() {
    final d = DateTime.now();
    return '${_weekdays[d.weekday - 1]}, ${d.day} ${_months[d.month - 1]}'
        .toUpperCase();
  }

  @override
  Widget build(BuildContext context) {
    final lib = context.lib;
    final reading = lib.sorted(
      lib.withStatus(ReadingStatus.reading),
      SortOption.recent,
    );
    final recent = lib.sorted(lib.books, SortOption.recent).take(6).toList();
    final picks = sampleBooks.where((b) => !lib.contains(b)).take(8).toList();

    return Constrained(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(Sp.page, Sp.md, Sp.page, Sp.xl),
        children: [
          const Logo(),
          const SizedBox(height: Sp.lg),
          Text(_today(), style: overline(context)),
          const SizedBox(height: Sp.xs + 2),
          Text(
            '${_greeting()},\n${lib.name}',
            style: serif(context, size: 30, height: 1.15),
          ),
```

```
          const SizedBox(height: Sp.lg),
          _GoalCard(onFind: () => widget.onGoTo(1)),
          SectionTitle(
            'Continue Reading',
            subtitle: reading.isEmpty ? null : '${reading.length} in progress',
          ),
          if (reading.isEmpty)
            Panel(
              child: Row(
                children: [
                  Icon(
                    Icons.auto_stories_outlined,
                    color: Theme.of(context).colorScheme.secondary,
                  ),
                  const SizedBox(width: Sp.md),
                  Expanded(
                    child: Text(
                      'Nothing in progress. Mark a book as "Currently Reading" to see it here.',
                      style: muted(context, size: 14).copyWith(height: 1.4),
                    ),
                  ),
                ],
              ),
            )
          else
            for (final b in reading)
              Padding(
                padding: const EdgeInsets.only(bottom: Sp.md - 4),
                child: _ContinueCard(b),
              ),

          SectionTitle(
            'Pick Up Your Next Read',
            subtitle: 'Sample suggestions',
          ),
          if (picks.isEmpty)
            Text(
              'You have added all of our sample picks. Try Discover for more.',
              style: muted(context, size: 14),
            )
          else
            BookCarousel(books: picks, onOpen: (b) => openDetails(context, b)),
          if (recent.isNotEmpty) ...[
            const SectionTitle('Recently Added'),
            BookCarousel(
              books: recent,
              tileWidth: 104,
              onOpen: (b) => openDetails(context, b),
            ),
          ],

          const SectionTitle('Reading Inspiration'),
          _QuoteCard(
            index: _quote,
            onNext: () =>
                setState(() => _quote = (_quote + 1) % sampleQuotes.length),
          ),
        ],
      ),
    );
  }
}

class _GoalCard extends StatelessWidget {
  const _GoalCard({required this.onFind});
  final VoidCallback onFind;

  @override
  Widget build(BuildContext context) {
    final lib = context.lib;
    final c = Theme.of(context).colorScheme;
    final left = lib.goal - lib.readCount;
    return Card(
      color: c.primaryContainer,
      child: Padding(
        padding: const EdgeInsets.all(Sp.md + 2),
        child: Row(
          children: [
            SizedBox(
```

```
                width: 68,
                height: 68,
                child: Stack(
                  alignment: Alignment.center,
                  children: [
                    SizedBox.expand(
                      child: CircularProgressIndicator(
                        value: lib.goalProgress,
                        strokeWidth: 6,
                        strokeCap: StrokeCap.round,
                        backgroundColor: c.onPrimaryContainer.withValues(
                          alpha: 0.18,
                        ),
                      ),
                    ),
                    Text(
                      '${(lib.goalProgress * 100).round()}%',
                      style: serif(
                        context,
                        size: 16,
                        color: c.onPrimaryContainer,
                      ),
                    ),
                  ],
                ),
              ),
              const SizedBox(width: Sp.md + 2),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      'YOUR READING GOAL',
                      style: overline(
                        context,
                        color: c.onPrimaryContainer.withValues(alpha: 0.75),
                      ),
                    ),
                    const SizedBox(height: 3),
                    Text(
                      '${lib.readCount} of ${lib.goal} books',
                      style: serif(
                        context,
                        size: 21,
                        color: c.onPrimaryContainer,
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      left <= 0 ? 'Goal reached. Well done!' : '$left more to go',
                      style: TextStyle(
                        fontSize: 13,
                        color: c.onPrimaryContainer.withValues(alpha: 0.85),
                      ),
                    ),
                  ],
                ),
              ),
              IconButton.outlined(
                tooltip: 'Find a book',
                onPressed: onFind,
                style: IconButton.styleFrom(
                  foregroundColor: c.onPrimaryContainer,
                  side: BorderSide(
                    color: c.onPrimaryContainer.withValues(alpha: 0.4),
                  ),
                ),
                icon: const Icon(Icons.search),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _ContinueCard extends StatelessWidget {
  const _ContinueCard(this.book);
  final Book book;
```

```
  @override
  Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: () => openDetails(context, book),
        child: Padding(
          padding: const EdgeInsets.all(Sp.md - 4),
          child: Row(
            children: [
              BookCover(book, width: 64),
              const SizedBox(width: Sp.md),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      book.title,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: serif(context, size: 16, height: 1.2),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      book.author,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: muted(context),
                    ),
                    const SizedBox(height: Sp.md - 2),
                    ProgressBar(book.progress / 100),
                    const SizedBox(height: Sp.sm - 2),
                    Text(
                      '${book.progress}% read',
                      style: muted(context, size: 12),
                    ),
                  ],
                ),
              ),
              Icon(
                Icons.chevron_right,
                color: Theme.of(context).colorScheme.onSurfaceVariant,
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _QuoteCard extends StatelessWidget {
  const _QuoteCard({required this.index, required this.onNext});
  final int index;
  final VoidCallback onNext;

  @override
  Widget build(BuildContext context) {
    final c = Theme.of(context).colorScheme;
    final quote = sampleQuotes[index % sampleQuotes.length];
    return Card(
      child: Padding(
        padding: const EdgeInsets.fromLTRB(Sp.lg, Sp.md, Sp.md, Sp.sm),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              '“',
              style: serif(context, size: 56, color: c.secondary, height: 1),
            ),
            Text(
              quote.$1,
              style: serif(
                context,
                size: 21,
                weight: FontWeight.w400,
                height: 1.4,
              ).copyWith(fontStyle: FontStyle.italic),
            ),
```

```
            const SizedBox(height: Sp.md),
            Row(
              children: [
                Container(width: 24, height: 1.5, color: c.secondary),
                const SizedBox(width: Sp.sm + 2),
                Expanded(
                  child: Text(quote.$2.toUpperCase(), style: overline(context)),
                ),
              ],
            ),
            const SizedBox(height: Sp.sm),
            Row(
              children: [
                Text('Sample quotes', style: muted(context, size: 12)),
                const Spacer(),
                TextButton.icon(
                  onPressed: onNext,
                  icon: const Icon(Icons.arrow_forward, size: 16),
                  iconAlignment: IconAlignment.end,
                  label: const Text('Next'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

OUTPUT

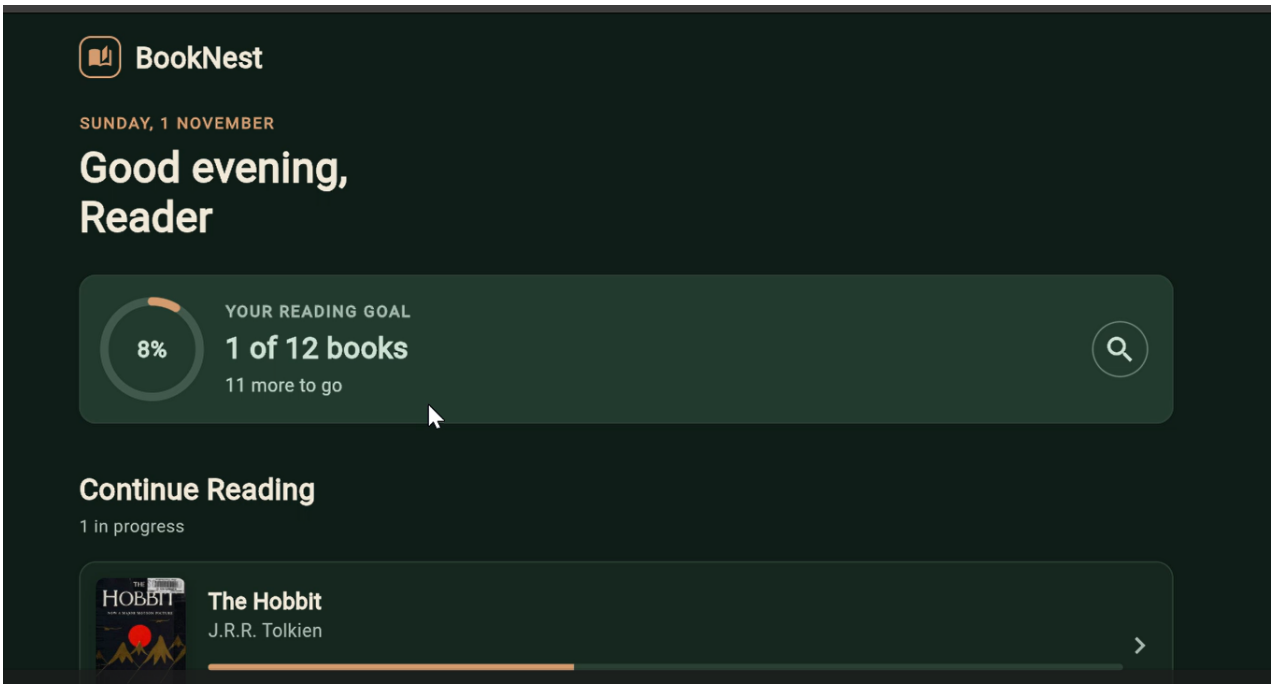


Figure 1: Home screen - greeting, reading-goal card and Continue Reading

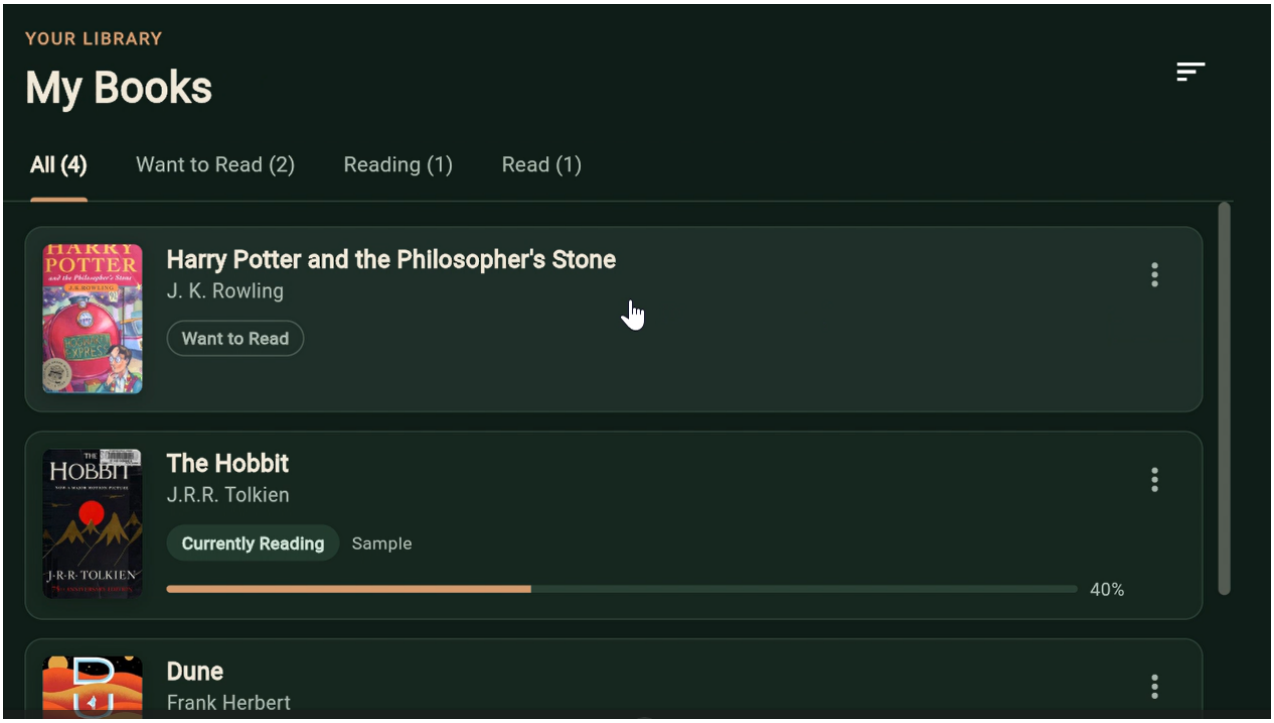


Figure 2: My Books - status tabs with counts, book cards and progress

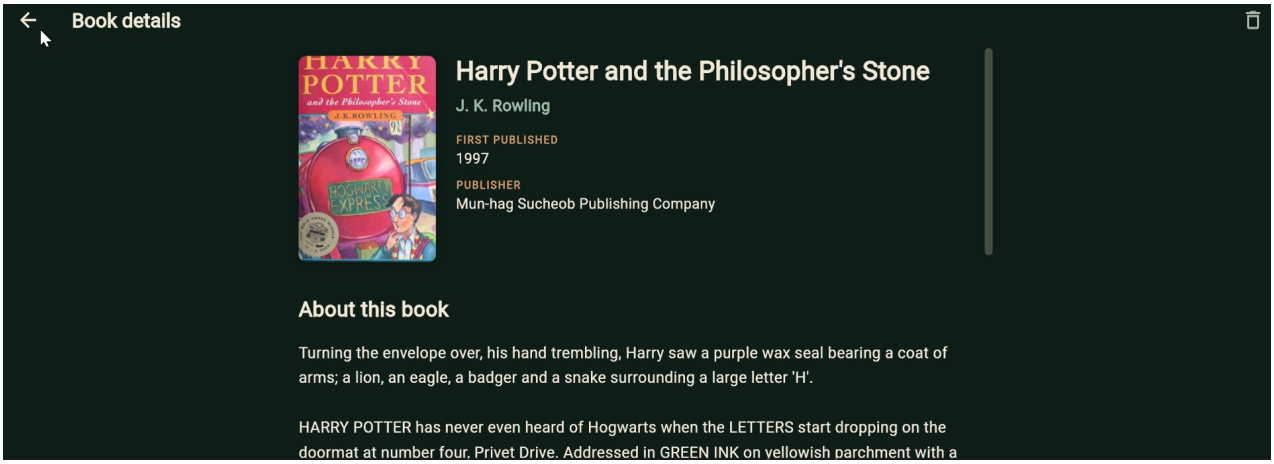


Figure 3: Book details - cover, author, publication facts and description

GITHUB Link:

https://github.com/balagamrisha/BookNest

Conclusion:

BookNest meets its goal of providing a simple, attractive and fully working book tracker without any backend. Users can discover books through the Open Library API, build a personal library, record their reading status, favorites, ratings and reviews, and follow a yearly reading goal. All counts and progress shown on Home, My Books and Profile are calculated from the stored library, so the screens always agree with each other.

The project shows practical use of Flutter concepts: Material 3 theming, responsive layouts, asynchronous networking with debouncing and error handling, JSON serialisation, local persistence and lightweight state management with *ChangeNotifier*. Reliability features such as offline sample data, graceful handling of missing images or corrupt stored data, and duplicate prevention make the app dependable in everyday use.

Possible future improvements include cloud backup and sync, reading statistics per year, barcode scanning to add books, and reading reminders.